

# Icarus: Numerical Analysis

## 5-Minute Presentation Script & Algorithm Reference

Topic: Root-Finding Algorithm Comparison Platform

Target Audience: Academic / Engineering Evaluators

Presenter: Mehran

Duration: 5 Minutes (Strict Timeframe)

Project URL: <https://github.com/lonemehran/Icarus>

*Abstract: Icarus is an interactive dashboard that parses mathematical functions, classifies their analytic properties, and runs six major root-finding solvers (Bisection, Secant, Regula Falsi, Illinois, Brent's, and Newton-Raphson) side-by-side. This guide outlines a 5-minute presentation script, convergence/divergence analysis, and engineering takeaways.*

# 1. The 5-Minute Presentation Script

## MINUTE 0:00 - 1:00 | Introduction & The Core Problem

"Good morning. Today, I am presenting Icarus, a comparative platform for numerical root-finding. Root-finding (solving  $f(x) = 0$ ) is central to optimization, physics simulations, and engineering model matching. Usually, textbooks present Newton-Raphson as the default choice due to its quadratic speed. However, in real-world application, Newton-Raphson frequently fails due to local extrema, flat regions, or corners. Icarus was built to help engineers analyze functions visually and choose the most robust method for their specific  $f(x)$  by comparing six major numerical solvers side-by-side. Let me show you how it works."

## MINUTE 1:00 - 2:30 | Technical Platform Walkthrough & Live Demo

"Our system includes a front-end using MathLive for raw, Desmos-style LaTeX inputs. The user does not need to categorize their function; the backend uses SymPy to parse the input, differentiate it symbolically, and run properties detection (checking continuity, differentiability, and poles). On the left, we input  $f(x)$ , specify an interval  $[a, b]$ , tolerance (e.g.,  $10^{-10}$ ), and select our solvers. When I hit 'Run Analysis', the backend evaluates the algorithms and responds in milliseconds. On the dashboard, we instantly see a Comparison Table ranked by accuracy and speed, an Iteration Bar Chart, and Logarithmic Error Decay Rates plotting how fast each method approaches the solution."

## MINUTE 2:30 - 4:00 | Why Methods Behave Differently: Convergence vs Divergence

"During the run, we observe major behavioral differences. For bracket-based methods like Bisection, Regula Falsi, Illinois, and Brent's, the root remains trapped, guaranteeing convergence if a sign change  $f(a)f(b) < 0$  exists. Bisection is slow but steady (linear convergence). Illinois improves on Regula Falsi's tendency to stagnate on one side of a curve by halving the function value. Newton-Raphson and Secant are 'open' methods; they start from a guess and have no bounds. If  $f'(x)$  is close to zero, or if the function is flat, Newton overshoots wildly - for  $x^{10} - 1$ , starting at  $x_0 = 0.5$  causes Newton to leap to  $x = 51.7$ , leading to temporary divergence. Brent's method is the hybrid champion: it attempts fast quadratic steps (IQI) but falls back to Bisection if the steps behave erratically."

## MINUTE 4:00 - 5:00 | Engineering Takeaways & Conclusion

"In conclusion, Icarus proves that there is no 'single best' root-finding method. For smooth, well-bracketed equations with cheap derivatives, Newton-Raphson is the fastest. But for complex, black-box, or singular equations, Brent's method represents the industry gold standard because it guarantees Bisection safety while delivering Secant speed. Icarus exposes these trade-offs visually, making algorithmic decisions empirical rather than theoretical. Thank you, and I am open to any questions."

## 2. Deep-Dive: Why Some Solvers Diverge

Root-finding algorithms fall into two broad families: Bracket methods and Open methods. Their fundamental design differences explain why some converge stably while others fail.

### A. Bracket Methods: Guaranteed Convergence

Methods: Bisection, Regula Falsi, Illinois, and Brent's Method.

- Convergence Mechanism: These methods require an initial interval  $[a, b]$  where  $f(a)f(b) < 0$ . Because real-valued continuous functions must pass through zero to change signs (Intermediate Value Theorem), the root is guaranteed to lie within the bounds. The bracket is iteratively shrunk.
- Divergence Risks: Zero risk of divergence on continuous functions, provided the initial bracket is valid. If  $f(a)f(b) > 0$ , the algorithms cannot begin. If the function contains a pole (singularity, like  $1/x = 0$ ), bracket methods will converge to the asymptote, mistaking the pole for a zero-crossing because of the sign change.

### B. Open Methods: Speed vs. Instability Risks

Methods: Newton-Raphson and Secant Method.

- Convergence Mechanism: Start from an initial guess (or guesses) and use slope information (analytically or via finite difference) to project where  $f(x) = 0$ . Newton-Raphson achieves quadratic convergence:  $e_{k+1}$  is approximately proportional to  $e_k^2$ . The number of correct decimal places doubles each iteration.
- Divergence Risks:
  1. Vanishing Derivatives: If  $f'(x)$  approaches 0, the step  $dx = -f(x)/f'(x)$  blows up. The algorithm projects the next guess to infinity (e.g. flat regions in high-degree polynomials like  $x^{10} - 1$ ).
  2. Non-Differentiability / Corners: If  $f(x)$  has a discontinuous derivative (e.g. corners in  $\text{abs}(x) - 2$ ), the tangent slope bounces back and forth across the corner, leading to endless oscillation between two states.
  3. Chaotic Inflection Points: If the initial guess is far from the root, the presence of local extrema can kick iterates into different basins of attraction, causing chaotic divergence or cycling.

### 3. Summary of Algorithms & Engineering Conclusions

| Method         | Type    | Convergence Order     | Optimal Scenario / Recommendation                    |
|----------------|---------|-----------------------|------------------------------------------------------|
| Bisection      | Bracket | Linear (Order 1.0)    | Extremely simple; use for safety fallback.           |
| Regula Falsi   | Bracket | Linear (stagnates)    | Poor choice; prone to slow single-side lock.         |
| Illinois       | Bracket | Superlinear (~1.44)   | Robust; resolves Regula Falsi stagnation.            |
| Brent's        | Hybrid  | Superlinear (~1.62)   | Default industry choice; fast and guaranteed.        |
| Newton-Raphson | Open    | Quadratic (Order 2.0) | Fastest; requires cheap, smooth derivatives.         |
| Secant         | Open    | Superlinear (~1.62)   | Fast; use when derivatives are expensive to compute. |

#### Key Engineering Conclusions

- Never default to Newton-Raphson blindly: The speed advantages are only realized close to the root. Always evaluate derivative availability and risk of zero slopes.
- Brent's Method is the gold standard: By combining Inverse Quadratic Interpolation (IQI), Secant, and Bisection, it switches strategies dynamically. Brent's method converges at worst like Bisection (guaranteed) and at best like IQI (superlinear/quadratic).
- Live properties detection helps avoid errors: Using analytic parsing tools like SymPy allows applications to flag discontinuities (e.g. sign changes with poles) before submitting variables to numerical solvers, avoiding infinite loops.